EXP09

CODE:

```
import torchimport torch.nn as nnimport torch.optim as optimfrom
torch.utils.data import TensorDataset, DataLoaderimport numpy as
npimport urllib.request
device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')print(f"Using device: {device}")
url = 'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt'file_path =
'shakespeare.txt'urllib.request.urlretrieve(url, file_path)
with open(file_path, 'r', encoding='utf-8') as f:
    text = f.read().lower()
chars = sorted(list(set(text)))vocab_size = len(chars)c2i = {c: i for
i, c in enumerate(chars)}i2c = {i: c for i, c in enumerate(chars)}
print(f"Corpus Length: {len(text)} characters")print(f"Unique
Characters: {vocab_size}")

seq_len = 40step = 3
X_data, y_data = [], []for i in range(0, len(text) - seq_len, step):
    input_seq = text[i:i + seq_len]
    target_char = text[i + seq_len]
    X_data.append([c2i[c] for c in input_seq])
    y_data.append(c2i[target_char])
X_tensor = torch.tensor(X_data, dtype=torch.long)y_tensor =
torch.tensor(y_data, dtype=torch.long)
batch_size = 128dataset = TensorDataset(X_tensor, y_tensor)dataloader
= DataLoader(dataset, batch_size=batch_size, shuffle=True)
class CharLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128):
        super(CharLSTM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        out = self.embedding(x)
        out, hidden = self.lstm(out, hidden)
        out = self.fc(out[:, -1, :])
        return out, hidden
model = CharLSTM(vocab_size, embed_dim=64,
hidden_dim=128).to(device)criterion = nn.CrossEntropyLoss()optimizer
= optim.Adam(model.parameters(), lr=0.002)
epochs = 5model.train()
```

```
for epoch in range(epochs):
    total_loss = 0.0
    for X_batch, y_batch in dataloader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        output, _ = model(X_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    print(f"Epoch [{epoch + 1}/{epochs}] - Loss: {avg_loss:.4f}")

def generate_text(model, seed, length=300, temperature=0.7):
    model.eval()
    seed_cleaned = seed.lower()

    if len(seed_cleaned) < seq_len:
        seed_cleaned = seed_cleaned.rjust(seq_len)

    seq_indices = [c2i.get(c, 0) for c in seed_cleaned[-seq_len:]]
    generated = seed

    with torch.no_grad():
        for _ in range(length):
            inp = torch.tensor([seq_indices[-seq_len:]],
dtype=torch.long).to(device)
            logits, _ = model(inp)

            logits = logits.squeeze(0) / temperature
            probs = torch.softmax(logits, dim=-1).cpu().numpy()

            next_idx = np.random.choice(vocab_size, p=probs)

            seq_indices.append(next_idx)
            generated += i2c[next_idx]

    return generated
seed_prompt = "shall i compare thee to a summer's
day?\n"generated_output = generate_text(model, seed=seed_prompt,
length=300, temperature=0.7)
```

```
print("\n--- Generated Text Output ---")print(generated_output)
```

OTUPUT:

```
Using device: cuda
Corpus Length: 1115394 characters
Unique Characters: 39
Epoch [1/5] - Loss: 1.9058
Epoch [2/5] - Loss: 1.6307
Epoch [3/5] - Loss: 1.5507
Epoch [4/5] - Loss: 1.5079
Epoch [5/5] - Loss: 1.4788

--- Generated Text Output ---
shall i compare thee to a summer's day?

orcentio:
no less of the propection,
forthen you case of worse her!

lucio:
you say, weath, with were
to me shillows and thou wert the field, master, so understores or all him.

benvolio:
thou accate one
or now? i stand the paties to his.

first neepling:
but, and, and all a man while i do bear doo
```